PEMROGRAMAN SISI SERVER

Minggu 1 — Backend Fundamentals & Development Environment

Memahami backend, client–server, HTTP, Python, Git, dan Docker

Suprayogi,M.Kom

Peta Pembelajaran Minggu 1–16

M1 Backend Fundamentals
Backend Fundamentals

M2 Architectural Patterns & System Design
Architectural Patterns & System Design

M3 Docker Containerization
Docker Containerization

M4 Django & ORM
Django & ORM

M5 Database Optimization
Database Optimization

M6 REST API
REST API

M7 Authentication & Authorization
Authentication & Authorization

M8 UTS
UTS

M9 NoSQL & Redis
NoSQL & Redis

M10 RabbitMQ & Celery
RabbitMQ & Celery

M11 Testing Strategies
Testing Strategies

M12 CI/CD & Deployment
CI/CD & Deployment

M13 System Integration & Performance
System Integration & Performance

M14 Security & Production Deployment
Security & Production Deployment

M15 Finalization & Presentation
Finalization & Presentation

M16 UAS
UAS

Tujuan Pembelajaran Minggu 1

- Menjelaskan peran backend dalam sebuah aplikasi web.
- Memahami alur client → server → database → response.
- Mengenali HTTP request, response, method, status code, dan JSON.
- Menyiapkan Python virtual environment dan menjalankan program sederhana.
- Memahami fungsi Git dan GitHub dalam pengelolaan source code.
- Memverifikasi Docker dan Docker Compose di komputer.

Metode Pembelajaran

Konsep + demo + latihan terbimbing

- Penjelasan konsep inti.
- Demo langsung: backend sederhana dan tools development.
- Guided lab: mahasiswa mengikuti langkah yang sama di komputer masing-masing.
- Diskusi kelas: menghubungkan konsep dengan aplikasi nyata.

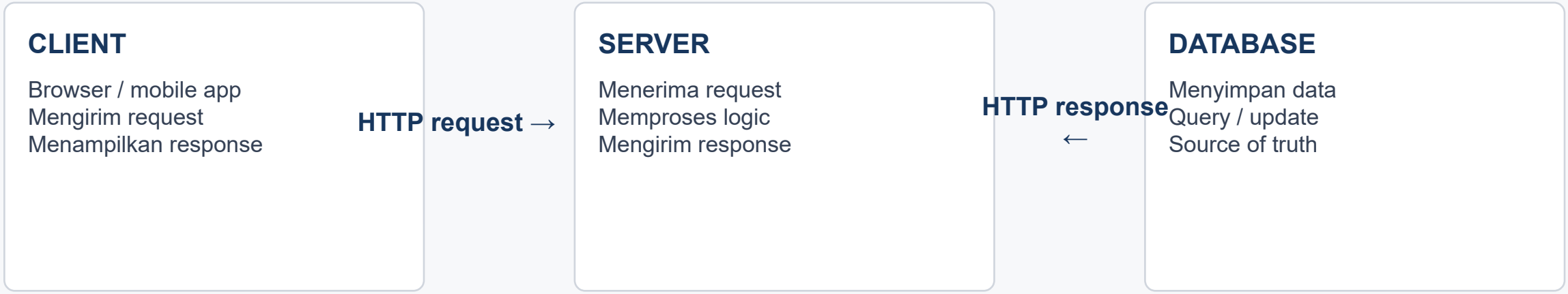
Apa Itu Backend?

- Backend adalah bagian aplikasi yang memproses request, menjalankan aturan bisnis, mengakses data, dan mengirim response.
- Backend berjalan di server dan biasanya tidak langsung terlihat oleh pengguna.
- Frontend menampilkan antarmuka; backend menyediakan data dan layanan yang dibutuhkan frontend.

Contoh

Login
→ validasi username/password
→ cek database
→ buat session/token
→ kirim response

Client–Server



Alur Request–Response

1. User melakukan aksi di browser.
2. Browser membentuk HTTP request.
3. Server menerima dan memvalidasi request.
4. Backend menjalankan business logic dan dapat membaca database.
5. Server mengirim HTTP response.
6. Client menampilkan hasil kepada user.

HTTP Request

Komponen penting

Method: GET, POST, PUT, PATCH, DELETE
URL / path
Headers
Query parameters
Body / payload

Contoh

GET /api/courses
POST /api/courses
PATCH /api/courses/12
DELETE /api/courses/12

Pertanyaan

Data apa yang dikirim?
Ke endpoint mana?
Method apa?
Apakah perlu authentication?

HTTP Response & Status Code

2xx — berhasil

200 OK
201 Created
202 Accepted
204 No Content

4xx — kesalahan client

400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found

5xx — kesalahan server

500 Internal Server Error
502 Bad Gateway
503 Service Unavailable

JSON sebagai Format Data API

- JSON adalah format teks yang umum dipakai untuk pertukaran data API.
- Struktur dasarnya berupa object { } dan array [ ].
- Nama field sebaiknya konsisten dan mudah dipahami.
- Contoh response: id, title, description, is_active.

Contoh response

```
{
  "id": 1,
  "title": "Python Backend",
  "is_active": true
}
```

Demo: Backend Tanpa Framework

- Tujuan demo bukan membangun aplikasi produksi.
- Kita hanya ingin melihat mekanisme dasar server.
- Python menyediakan module sederhana untuk menerima request dan mengirim response.
- Setelah konsep dipahami, framework seperti Django akan membuat pekerjaan lebih terstruktur.

Ide sederhana

Browser
↓
Python HTTP Server
↓
Response text / JSON

Python untuk Development Backend

- Pastikan Python dapat dipanggil dari terminal.
- Gunakan virtual environment agar dependency proyek terisolasi.
- Biasakan menjalankan perintah dari folder proyek.
- Jangan mencampur dependency antar proyek.

Virtual Environment

- Virtual environment adalah lingkungan Python terpisah untuk satu proyek.
- Dependency proyek A tidak harus sama dengan proyek B.
- Aktivasi environment dilakukan sebelum instalasi dan eksekusi dependency proyek.
- Untuk mata kuliah ini, konsep environment akan dipakai lagi saat Django dan Docker.

Pola kerja

```
project/
├── .venv/
├── manage.py
├── requirements.txt
└── app/
```

Dependency Management

- Package Python dipasang sesuai kebutuhan proyek.
- requirements.txt membantu mencatat dependency yang dibutuhkan.
- Versi dependency perlu diperhatikan agar environment dapat direproduksi.
- Pada deployment, dependency yang sama harus dapat dipasang kembali.

Git: Mengelola Perubahan Source Code

- Git mencatat perubahan file dari waktu ke waktu.
- Commit menjadi titik perubahan yang dapat dilacak.
- Branch membantu memisahkan pekerjaan.
- Git bukan sekadar tempat menyimpan file; Git menyimpan riwayat perubahan.

GitHub: Kolaborasi & Repository

- GitHub menjadi remote repository yang dapat diakses tim.
- Repository menyimpan source code, commit, branch, issue, dan dokumentasi.
- Push mengirim commit lokal ke remote.
- Pull mengambil perubahan dari remote ke lokal.

Latihan di Kelas: Git

- Buat folder proyek sederhana.
- Inisialisasi repository Git.
- Buat satu file Python sederhana.
- Lakukan commit pertama.
- Jika repository GitHub sudah tersedia, praktikkan push.

Mengapa Docker?

- Docker membantu menjalankan aplikasi dalam environment yang lebih konsisten.
- Dependency aplikasi dapat dikemas bersama konfigurasi runtime.
- Container memudahkan proses development, testing, dan deployment.
- Pada minggu berikutnya Docker akan dipakai bersama Django, database, Redis, dan RabbitMQ.

Image vs Container

IMAGE

Template / paket aplikasi
Berisi filesystem dan dependency
Dapat digunakan untuk membuat container

CONTAINER

Instance yang sedang berjalan
Memiliki process dan network sendiri
Dapat dibuat / dihentikan / dihapus

Docker Compose

- Compose digunakan untuk mendefinisikan beberapa service aplikasi.
- Contoh service yang akan kita gunakan pada semester ini: Django, database, Redis, RabbitMQ.
- Compose membantu membuat network antar-service secara konsisten.
- Nama service dapat digunakan sebagai hostname di dalam network Compose.

Verifikasi Docker di Kelas

- Cek Docker version.
- Cek Docker Compose version.
- Jalankan container sederhana.
- Pastikan container dapat dihentikan dan dihapus kembali.
- Jika ada error, fokus dulu pada pesan error dan environment komputer.

Development Environment yang Sehat

Code

Editor / IDE
Terminal
Git

Runtime

Python
Virtual environment
Dependency

Infrastructure

Docker
Compose
Database / service

Kesalahan Umum di Awal

- Menganggap backend = database saja.
- Tidak memahami request dan response.
- Menjalankan dependency global tanpa environment terisolasi.
- Tidak melakukan commit secara berkala.
- Mengira Docker otomatis memperbaiki semua masalah environment.

Contoh Project Simple LMS

- Capstone semester menggunakan contoh Simple LMS sebagai konteks aplikasi.
- Data utama: Course dan Lesson.
- Backend akan berkembang bertahap dari konsep dasar menuju API, authentication, cache, async task, testing, CI/CD, dan deployment.
- Minggu 1 hanya membangun fondasi pemahaman dan environment.

Demo Mini Backend

Yang sudah dicoba

Backend sederhana tanpa framework
Request → processing → response
Konsep HTTP dan JSON

Belum masuk

Django
ORM
REST API
Authentication
Redis / Celery

Ringkasan Minggu 1

- Backend menerima request, memproses logic, dan mengirim response.
- HTTP menjadi dasar komunikasi client–server.
- Python dan virtual environment menyiapkan runtime development.
- Git/GitHub mengelola source code dan riwayat perubahan.
- Docker membantu konsistensi environment.

Pertemuan Minggu 2

- Minggu 2: Architectural Patterns & System Design.
- Kita mulai membahas bagaimana komponen backend disusun.
- Topik akan bergerak dari sekadar “server berjalan” menjadi “sistem dirancang”.
- Simple LMS menjadi contoh untuk mempraktikkan struktur sistem.